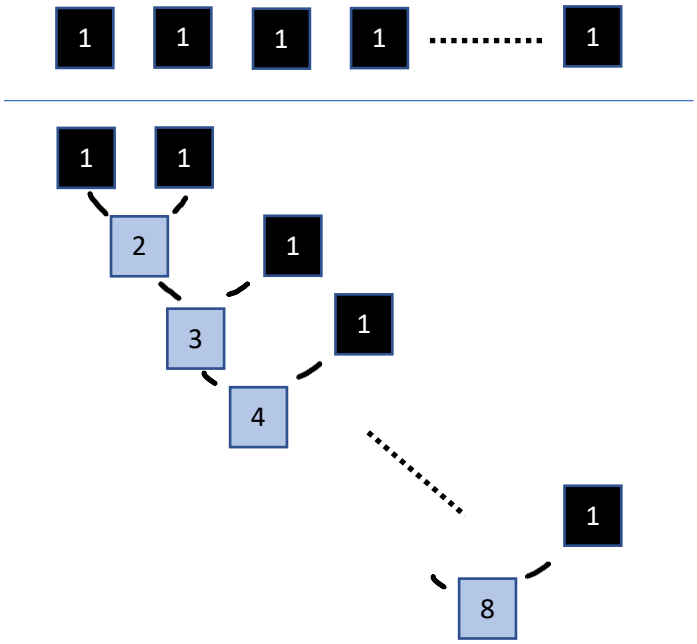
LUCRAREA 4

IMPLEMENTAREA ALGORITMILOR PARALELI. REDUCEREA PARALELA. SORTAREA. RECURSIVITATE.

1. REDUCEREA PARALELA

Algoritmii de reducere reprezinta acea categorie de algoritmi care genereaza la iesire o singura valoare pe baza unui vector de valori de intrare (reducerea). Cateva exemple de algoritmi de reducere: suma elementelor unui vector, calculul valorii minime sau maxime a elementelor unui vector.

In continuare vom exemplifica algoritmul serial de calcul al sumei elementelor unui vector de dimensiune ***N***:




```
using System;
using System.Diagnostics;

namespace L4
{
    internal class Program
    {
        public static int N = 16;//N items

        public static double[] a;//array of items
        public static double sTemp;

        static void Main(string[] args)
        {
```

```
            a = new double[N];

            int i;
            for (i = 0; i < N; i++)
            {
                a[i] = 1;
            }
            Console.Write("A: ");
            Print(a);

            //sequencial reduction
            Stopwatch timerSeq = new Stopwatch();
            timerSeq.Start();
            double s = ReductionSeq();
            timerSeq.Stop();
            Console.WriteLine("Sum(A)={0}; Seq time = {1} ms", s, timerSeq.Elapsed.TotalMilliseconds);

            //parallel reduction V1
            Stopwatch timerPar1 = new Stopwatch();
            timerSeq.Start();
            s = ReductionParV1();
            timerSeq.Stop();
            Console.WriteLine("Sum(A)={0}; ParV1 time = {1} ms", s, timerPar1.Elapsed.TotalMilliseconds);


            Console.WriteLine("Press any key for exit...");
            Console.ReadKey();
        }
        public static void Print(double[] m)
        {
            int i;
            for (i = 0; i < ((N < 8) ? N : 8); i++)
            {
                Console.Write(string.Format("{0} ", m[i]));
            }
            Console.WriteLine();
        }
        public static double ReductionSeq()
        {
            double s = a[0];
            int i;
            for (i = 1; i < N; i++)
            {
                s += a[i];
            }
            return s;
        }
    }
}
```

Complexitatea implementarii seriale este ***O(N)***. In continuare vom implementa algoritmul de reducere paralela folosind diverse strategii de paralelizare.

Implementarea multithreading a sumei elementelor unui vector se poate realiza folosind o variabila globala care sa memoreze valoarea sumei ***sTemp***, si un numar de ***N*** threaduri, fiecare adunand elementul corespunzator din vector la aceasta variabila globala. Analiza dependentelor unei astfel de implementari va arata ca datele de intrare sunt partial independente, fiecare thread va citi cate un element separat din vector, respectiv partial dependente, fiecare thread citeste valoarea sumei partiale si o incrementeaza cu valoarea propriului element. Variabila partajata ***sTemp*** este citita si modificata concurent de catre toate threadurile:

```
using System;
using System.Diagnostics;
using System.Threading;

namespace L4
{
    internal class Program
```

```
    {
        public static int N = 128;//N items

        public static double[] a; //array of items
        public static double sTemp; //shared
        static void Main(string[] args)
        {
             ....
            //parallel reduction V1
            Stopwatch timerPar1 = new Stopwatch();
            timerPar1.Start();
            s = ReductionParV1();
            timerPar1.Stop();
            Console.WriteLine("Sum(A)={0}; ParV1 time = {1} ms", s, timerPar1.Elapsed.TotalMilliseconds);


            Console.WriteLine("Press any key for exit...");
            Console.ReadKey();
        }
        ...
        public static double ReductionParV1()
        {
            sTemp = 0;

            Thread[] childThreads = new Thread[N];
            int i;
            for (i = 0; i < N; i++)
            {
                childThreads[i] = new Thread(ThreadFunctionV1);
                childThreads[i].Start(i);
            }
            for (i = 0; i < N; i++)
            {
                childThreads[i].Join();
            }

            return sTemp;
        }
        public static void ThreadFunctionV1(object parameter)
        {
            int idx = (int)parameter;
            sTemp += a[idx];
        }
    }
}
```

EXERCIȚIUL 1:

Implementati versiunea paralela (V1). Masurati timpul de executie si evaluati corectitudinea rezultatului raportat la implementarea seriala.

EXERCIȚIUL 2:

Completati versiunea V1 a implementarii astfel incat sa se asigure executia corecta (se poate utiliza o variabila de tip mutex pentru accesul la variabila partajata ***sTemp***).

Implementarea paralela V1 de mai sus reprezinta o implementare cu granularitatea cea mai fina (numarul de threaduri este egal cu numarul de elemente). Desi aceasta implementare poate calcula corect suma elementelor unui vector (conform implementarii din Exercitiul 2), din punct de vedere al gradului de paralelism se poate observa ca executia este una seriala, accesul la variabila globala in care se acumuleaza sumele partiale fiind serializat.

In continuare vom prezenta o implementare care permite setarea granularitatii: cele ***N*** elemente se impart in ***P*** partitii independente de ***N/P*** elemente, fiecare partitie se atribuie unui thread (***P*** threaduri in total), fiecare thread calculeaza serial suma partiala, la final threadul principal calculeaza suma sumelor partiale:

```
using System;
using System.Diagnostics;
using System.Threading;

namespace L4
{
    internal class Program
    {
        public static int N = 128;//N items
        public static int P = 16;//P threads
        public static double[] pSum; //partial sums

        public static double[] a; //array of items
        public static double sTemp; //shared
        static void Main(string[] args)
        {
            ...
            //parallel reduction V2
            Stopwatch timerPar2 = new Stopwatch();
            timerPar2.Start();
            s = ReductionParV2();
            timerPar2.Stop();
            Console.WriteLine("Sum(A)={0}; ParV2 time = {1} ms", s, timerPar2.Elapsed.TotalMilliseconds);

            Console.WriteLine("Press any key for exit...");
            Console.ReadKey();
        }
        ...
        public static double ReductionParV2()
        {
            sTemp = 0;
            pSum = new double[P];
            Thread[] childThreads = new Thread[P];
            int i;
            for (i = 0; i < P; i++)
            {
                childThreads[i] = new Thread(ThreadFunctionV2);
                childThreads[i].Start(i);
            }
            for (i = 0; i < P; i++)
            {
                childThreads[i].Join();
                sTemp += pSum[i];
            }

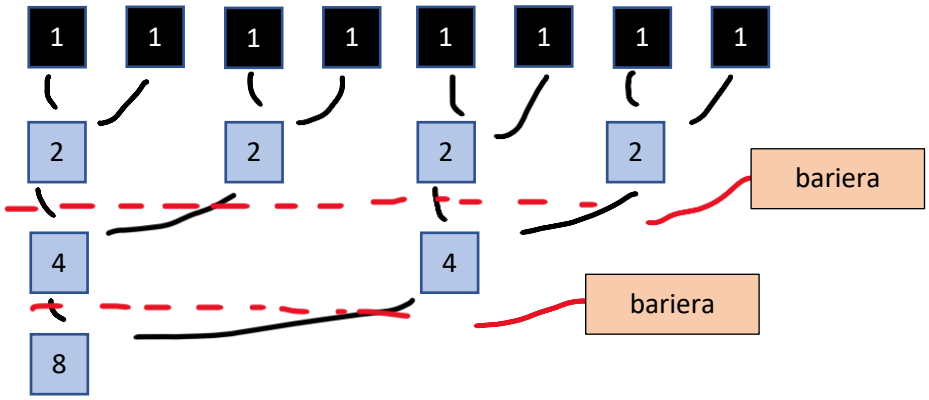
            return sTemp;
        }
        public static void ThreadFunctionV2(object parameter)
        {
            int idx = (int)parameter;
            pSum[idx] = 0;
            for (int i = (N/P)*idx; i < (N/P)*(idx + 1); i++)
            {
                pSum[idx] += a[i];
            }
        }
    }
}
```

Complexitatea implementarii paralele in aceasta implementare este ***O(N/P)*** la care se adauga complexitatea calculului sumei finale pe baza sumelor partiale ***O(P)***.

EXERCIȚIUL 3:

Implementati versiunea paralela (V2). Masurati timpul de executie si evaluati corectitudinea rezultatului raportat la implementarea seriala.

O implementare mai eficienta, cel putin din punctul de vedere al analizei complexitatii, de complexitate *O(Log(N))*, este prezentata in continuare. Aceasta implementare presupune lansarea a *N/2* threaduri, fiecare thread calculand o suma partiala a doua elemente. Algoritmul are *log(N)* pasi, in fiecare pas rezultand jumatate de valori sume intermediare fata de pasul anterior: in primul pas se genereaza *N/2* sume partiale, in pasul doi *N/2/2=N/4* sume partiale, .. , iar in pasul *log(N)* se calculeaza suma totala pe baza ultimelor doua sume partiale. Acest algoritm are nevoie de un punct de sincronizare – bariera (*barrier*), prin care toate threadurile sa se sincronizeze si astfel sa se asigure ca toate threadurile si-au calculat sumele partiale din pasul anterior pentru a putea incepe calculul din pasul curent.




```
using System;
using System.Diagnostics;
using System.Threading;

namespace L4
{
    internal class Program
    {
        public static int N = 128;//N items

        public static Barrier barier;
        static void Main(string[] args)
        {

            ...

            //parallel reduction V3
            Stopwatch timerPar3 = new Stopwatch();
            timerPar3.Start();
            s = ReductionParV3();
            timerPar3.Stop();
            Console.WriteLine("Sum(A)={0}; ParV3 time = {1} ms", s, timerPar3.Elapsed.TotalMilliseconds);

            Console.WriteLine("Press any key for exit...");
            Console.ReadKey();
        }
        ...

        public static double ReductionParV3()
        {
            sTemp = 0;
            Thread[] childThreads = new Thread[N/2];
            barier = new Barrier(N / 2);
            int i;
            for (i = 0; i < N / 2; i++)
            {
                childThreads[i] = new Thread(ThreadFunctionV3);
```

```
                childThreads[i].Start(2*i);
            }
            for (i = 0; i < N / 2; i++)
            {
                childThreads[i].Join();
            }
            sTemp = a[0];
            return sTemp;
        }
        public static void ThreadFunctionV3(object parameter)
        {
            int idx = (int)parameter;
            for(int i = 1; i <= N/2; i *= 2)
            {
                barier.SignalAndWait();
                if (idx % (2 * i) == 0)
                    a[idx] += a[idx + i];
            }
        }
    }
}
```

EXERCIȚIUL 4:

Implementati versiunea paralela (V3). Masurati timpul de executie si evaluati corectitudinea rezultatului raportat la implementarea seriala.

Fiecare din implementarile anterioare, V2 sau V3, reprezinta cate o solutie pentru problema implementarii paralele a algoritmului de reducere. Versiunea V3 asigura gradul maxim de paralelism, avand granularitatea cea mai mica, insa, asa cum rezulta si din masuratorile timpului de executie, nu reprezinta o solutie eficienta practic pe sisteme cu un grad redus de paralelism (precum sistemele CPU) desi dpdv teoretic are complexitatea cea mai mica ***O(Log(N))***. Versiunea V2 asigura un grad flexibil de paralelism dar contine o componenta seriala care afecteaza gradul de paralelism total. Cele doua metode se pot implementa intr-o versiune compusa V2+V3: fiecare thread calculeaza suma partiala a unei partitii N/P conform V2, si sumele partiale sunt reduse la suma totala folosind V3. Astfel se obtine o complexitate a implementarii intre cele doua versiuni V2 si V3,

Complexitatea implementarii paralele in aceasta implementare este ***O(N/P)*** la care se adauga complexitatea calculului sumei finale pe baza sumelor partiale ***O(log(P))***

$$O(N/P) + O(P) > O(N/P) + O(\log(P)) > O(Log(N))$$

EXERCIȚIUL 5:

Implementati versiunea paralela (V4) combinand versiunile V2 si V3 anterioare. Masurati timpul de executie si evaluati corectitudinea rezultatului raportat la implementarea seriala.

O ultima versiune de implementare a algoritmului de reducere paralela reprezinta o variatie fata de implementarea V3, fara a mai utiliza un punct de sincronizare tip bariera (utilizand clasa ***Barrier***) ci folosind generarea recursiva a threadurlor:

```
using System;
using System.Diagnostics;
```

```
using System.Threading;


namespace L4
{
    public class ThreadParam
    {
        public int pos; //position in array
        public int len; //partition length
    }
    internal class Program
    {
        public static int N = 128;//N items
        ...
        static void Main(string[] args)
        {
            ...
            //parallel reduction V5 - recursion
            Stopwatch timerPar5 = new Stopwatch();
            timerPar5.Start();
            s = ReductionParV5();
            timerPar5.Stop();
            Console.WriteLine("Sum(A)={0}; ParV5 time = {1} ms", s, timerPar5.Elapsed.TotalMilliseconds);

            Console.WriteLine("Press any key for exit...");
            Console.ReadKey();
        }
        ...
        public static double ReductionParV5()
        {
            sTemp = 0;
            Thread[] childThreads = new Thread[2];
            int i;
            int j = 0;
            for (i = 0; i < N; i+= N/2, j++)
            {
                childThreads[j] = new Thread(ThreadFunctionV5);
                childThreads[j].Start(new ThreadParam { pos = i, len = N / 2 });
            }
            for (i = 0; i < 2; i++)
            {
                childThreads[i].Join();
            }
            sTemp = a[0] + a[N/2];
            return sTemp;
        }
        public static void ThreadFunctionV5(object parameter)
        {
            var p = (ThreadParam)parameter;
            if(p.len > 2)
            {
                Thread[] childThreads = new Thread[2];
                int i;
                int j = 0;
                for (i = 0; i < p.len; i += p.len / 2, j++)
                {
                    childThreads[j] = new Thread(ThreadFunctionV5);
                    childThreads[j].Start(new ThreadParam { pos = i, len = p.len / 2 });
                }
                for (i = 0; i < 2; i++)
                {
                    childThreads[i].Join();
                }
                a[p.pos] += a[p.pos + p.len / 2];
            }
            else
            {
                a[p.pos] += a[p.pos + 1];
            }
        }
    }
}
```

EXERCIȚIUL 6:

Implementati versiunea paralela (V5). Masurati timpul de executie si evaluati corectitudinea rezultatului raportat la implementarea seriala.

EXERCIȚIUL 7:

Implementati versiunea seriala si paralela a calculului valorii minime respectiv maxime a elementelor unui vector. Calculati media elementelor unui vector.

BIBLIOGRAFIE

[1]. PRINCIPIILE CALCULULUI PARALEL, Felicia Ionescu, EDITURA TEHNICĂ, București, 1999

[2]. C# Programming Guide – Thread Class https://docs.microsoft.com/en-us/dotnet/api/system.threading.thread?view=netframework-4.8